OWASP Top 10: A02:2021 – Cryptographic Failures

Course: Web Application Security

Goal: Understand the concept, impact, detection, exploitation, and mitigation of Cryptographic Failures—failures related to cryptography (or lack thereof) which often lead to exposure of sensitive data.

Presentation of the Vulnerability

What Are Cryptographic Failures?

Cryptographic failures refer to the improper use, implementation, or management of cryptographic systems. These issues often result in unauthorized exposure of sensitive data like passwords, credit card numbers, or personal records. In the OWASP Top 10 – 2021, this category replaced the broader 'sensitive data exposure' from the 2017 list, with a sharper focus on the misuse or failure of cryptographic mechanisms.

Cryptographic Failures encompass failures to properly protect sensitive data because of weak or misused cryptography, poor key management, or missing transport encryption—leading to exposure, tampering or impersonation.

OWASP Ranking

A02:2021

Ranked **#2** in the 2021 edition of the OWASP Top 10, shifting up one position to #2, previously known as Sensitive Data Exposure.

CIA Impact: Understanding the Damage

Cryptographic Failures vulnerabilities pose significant risks across the CIA triad—Confidentiality, Integrity, and Availability. Understanding these impacts helps prioritize security efforts and communicate risk to stakeholders.

Impact Area	Level	Explanation
Confidentiality	**High**	Sensitive data can be read by attackers through cleartext transmission, weak encryption, or exposed keys. This includes credentials, PII, financial info, and business secrets.
Integrity	**Medium**	Data may be modified or forged when authentication mechanisms fail or when weak crypto allows tampering with encrypted data without detection.
Availability	**Low**	Usually not directly impacted by cryptographic failures, though in some cases expired certificates or broken key management can cause service outages.

Deep Dive: Understanding Cryptographic Failures

Data Protection Failures

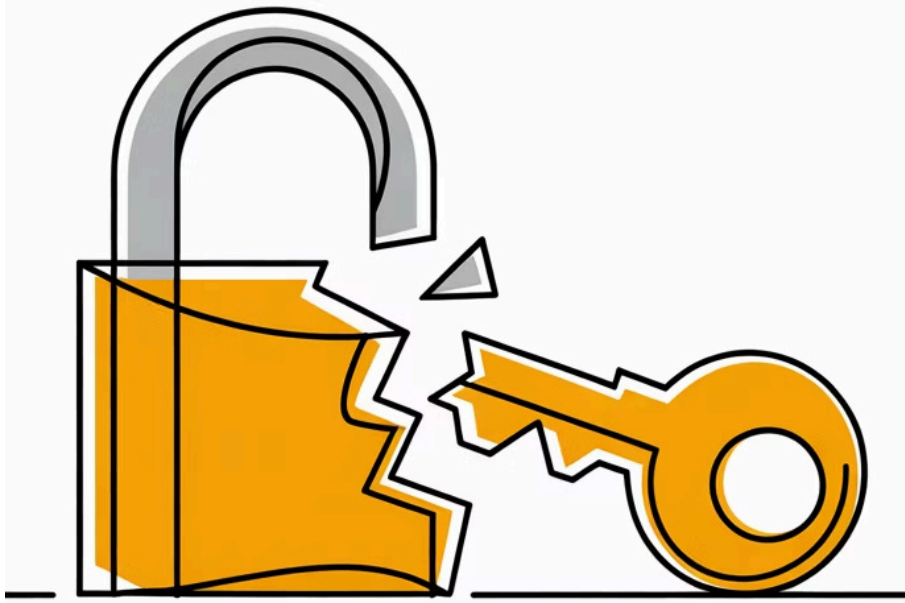
This vulnerability family focuses on *data protection, authentication & authorization failures* affecting sensitive data both at rest and in transit.

Attack Sources

Threats come from external attackers (MITM, network sniffing), internal actors (leaked keys), or supply-chain (vulnerable libs).

Attack Targets

Vulnerable components include backend (storage, DB), API servers (JWT/tokens), communication layers (TLS), config files, and CI/CD secrets.



Types of Cryptographic Failures

Cryptographic failures manifest in several distinct patterns. Understanding these categories helps security professionals identify and remediate weaknesses systematically.

1

Cleartext in Transit

Web applications sending login credentials, session tokens, or sensitive data over HTTP instead of HTTPS, allowing MITM attackers to steal information.

2

Weak Hashing for Passwords

Applications storing passwords with MD5, unsalted SHA-1, or other fast/broken hashing algorithms that enable rapid offline cracking attacks.

3

Poor At-Rest Encryption

Data encrypted using insecure modes like AES-ECB that reveal patterns, or missing encryption entirely for sensitive stored data.

4

Leaked or weak secrets

Usage of default or weak secrets, exposition of sensitive data leaking secrets (git, .env, application crash leaking code, etc.)

Example Scenario 1: Cleartext in Transit

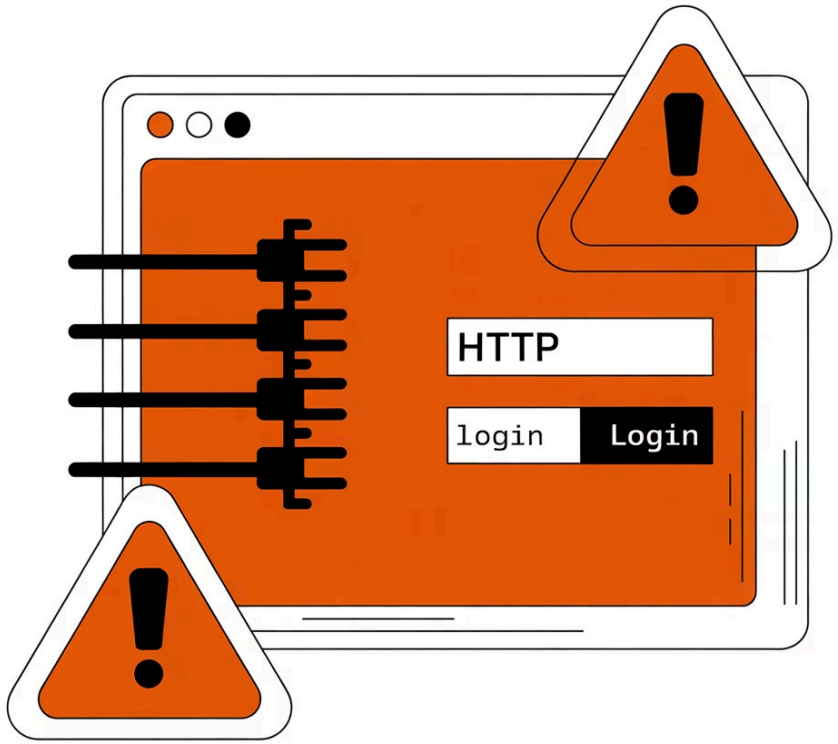
The Vulnerability

A web application sends login credentials and session tokens over HTTP instead of HTTPS. While the application may have some security measures, it fails to encrypt data during transmission.

An attacker performing a man-in-the-middle (MITM) attack can intercept network traffic and steal credentials and session cookies in cleartext—complete with usernames, passwords, and active session tokens.

Why It Happens

The server doesn't enforce TLS encryption or allows HTTP fallback. Developers may disable HTTPS in development and forget to re-enable it in production.



Example Scenario 2: Weak Password Hashing

Fast Offline Cracking

A web application stores user passwords hashed with MD5 or unsalted SHA-1. When the password database is compromised through a separate vulnerability or misconfiguration, attackers can use rainbow tables or powerful hardware to rapidly crack the hashes.

If the password database is ever leaked, through misconfiguration or an unrelated vulnerability, attackers can use precomputed rainbow tables to quickly crack the hashes and recover plaintext passwords. Key issue: Fast, unsalted hashes make it trivial for attackers to reverse passwords after a breach.

Database Breach

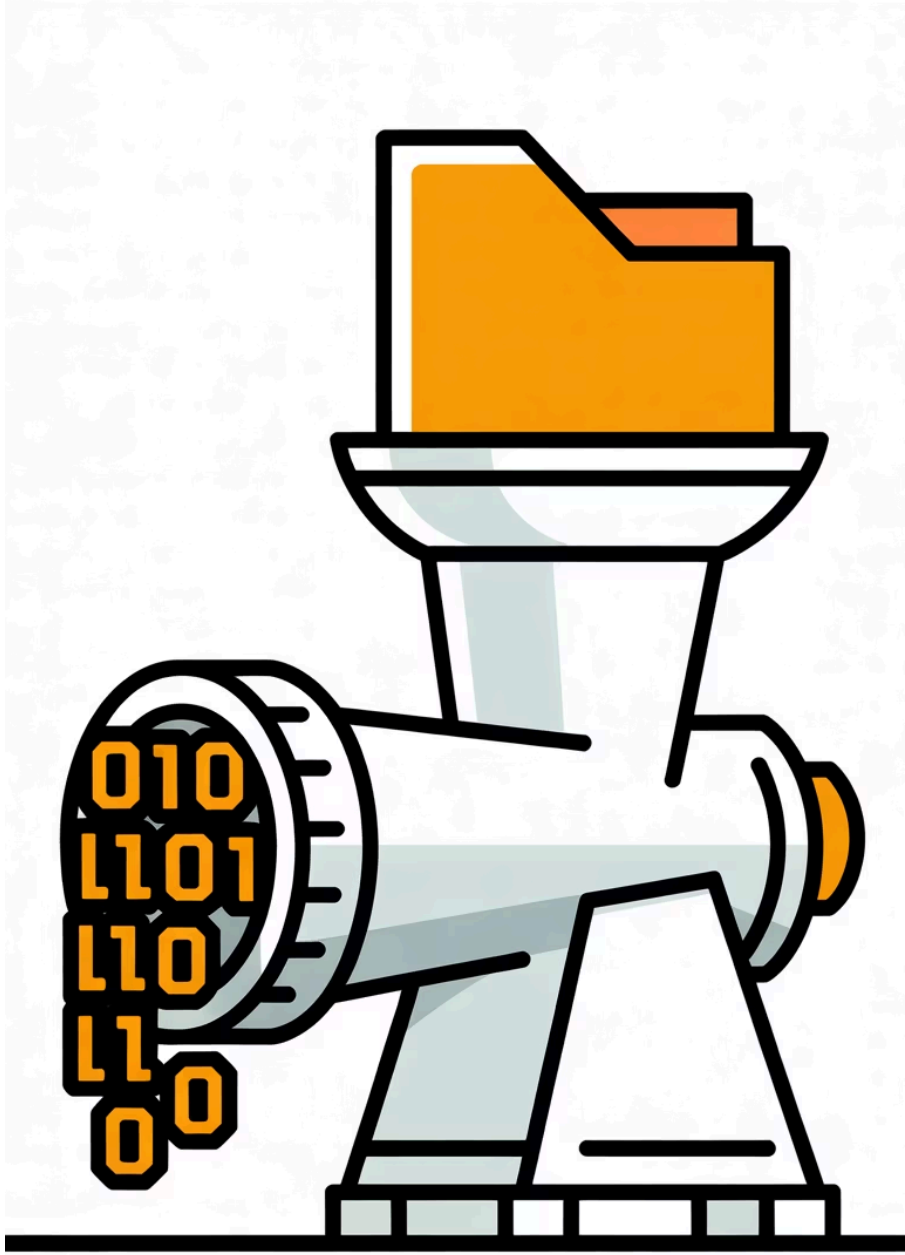
Attacker gains access to user database containing weakly hashed passwords

Offline Cracking

Using rainbow tables or GPU-accelerated cracking tools like Hashcat

Account Takeover

Recovered plaintext passwords used to access user accounts directly



Discovering the Vulnerability: Where to Look

Identifying cryptographic vulnerabilities requires systematic investigation across multiple attack surfaces. Security professionals should examine these key areas during penetration testing and code review.

Network Layer

HTTP endpoints, WebSocket traffic, API gateways. Look for any unencrypted data transmission or weak TLS configurations.

Backend Storage

Database backups, cloud storage, logs. Check for unencrypted sensitive data, weak encryption modes, or exposed configuration files.

Source Code & Configs

Look for secrets in code, deployment pipelines, environment variables. Search for hardcoded keys or credentials in repositories.

Token Handling

JWT verification code, token libraries, middleware. Check for insecure JWT validation or acceptance of "alg":"none" tokens.

TLS Configuration

Server TLS settings, certificate validation, HSTS headers. Look for weak ciphers, missing security headers, or certificate issues.

Crypto Libraries

Custom crypto implementations, deprecated algorithms, improper use of cryptographic functions in code.

Recognizing Warning Signs

Certain indicators strongly suggest the presence of cryptographic vulnerabilities. Train your security awareness to recognize these red flags during testing.

Plaintext Credentials

Login credentials or sensitive data visible in network captures or transmitted over unencrypted channels.

Deprecated Algorithms

Use of deprecated/weak algorithms in code (MD5, SHA-1, DES, RC4, ECB). Algorithms such as MD5, SHA1, and DES are considered cryptographically broken. Any 32 or 40 chars strings in hexa is a good sign. Also look for unsecure random source not suitable for crypto usage.

Missing TLS

Missing TLS or insecure TLS settings seen in testssl.sh/browser warnings. HTTP endpoints serving sensitive content.

Exposed Secrets

Presence of private keys, secrets or credentials in repos (Gitleaks alerts). Are crypto keys checked into source code repositories?

Weak JWT Validation

JWT verification that doesn't specify allowed algorithms or accepts tokens with "alg":"none" header values.

Verification Techniques

Manual Testing Methods

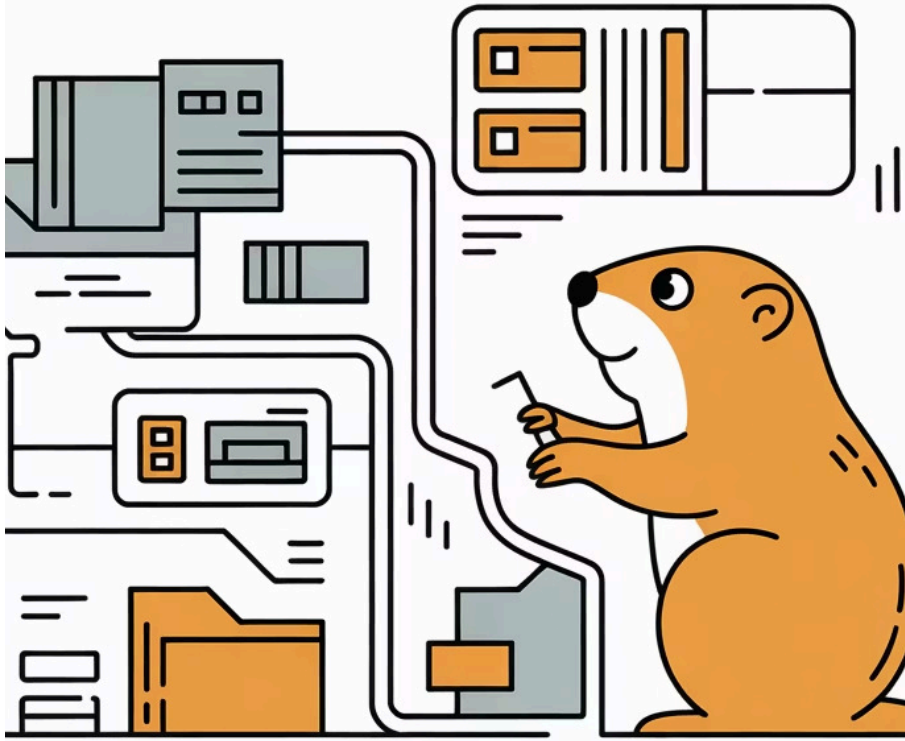
- **Network Interception:** Intercept traffic with tcpdump or Wireshark to see plaintext transport
- **Code Review:** Review code for crypto usage: custom crypto, manual implementations, wrong modes
- **JWT Testing:** Attempt to modify a JWT to alg:none and resend to see if accepted
- **Secret Scanning:** Inspect backup files, config files, Docker images for hardcoded secrets

Automated Testing

- **TLS Scanning:** Use testssl.sh or sslscan to check TLS configuration and identify weak ciphers
- **Path guessing:** Use tools such as gobuster to look for usual configuration files, such as .env and .git, as well a unsecure subdomain.

Hash Analysis

- **Extract password hashes** from database dumps and test with Hashcat/John the Ripper
- **Verify encryption modes** aren't revealing patterns (ECB mode detection)



Essential Testing Tools

Professional security testing requires the right tools. These industry-standard applications help identify and exploit cryptographic vulnerabilities effectively.Burp Suite / OWASP ZAP

Zed Attack Proxy/Burp

Intercept & modify HTTP(S) requests for JWT tests, analyze TLS configurations, and test for cleartext data transmission.

Wireshark

Capture network traffic to identify cleartext transmission of sensitive data and analyze network-level cryptographic failures.

testssl.sh / sslscan

TLS configuration checks to identify weak ciphers, missing security headers, and certificate validation issues.

Hashcat / John the Ripper

Test weak hashes by attempting to crack password databases extracted during security assessments.

Gitleaks / TruffleHog

Detect secrets in repositories by scanning Git history for hardcoded keys, passwords, and cryptographic credentials.

jwt-tool / jwt.io

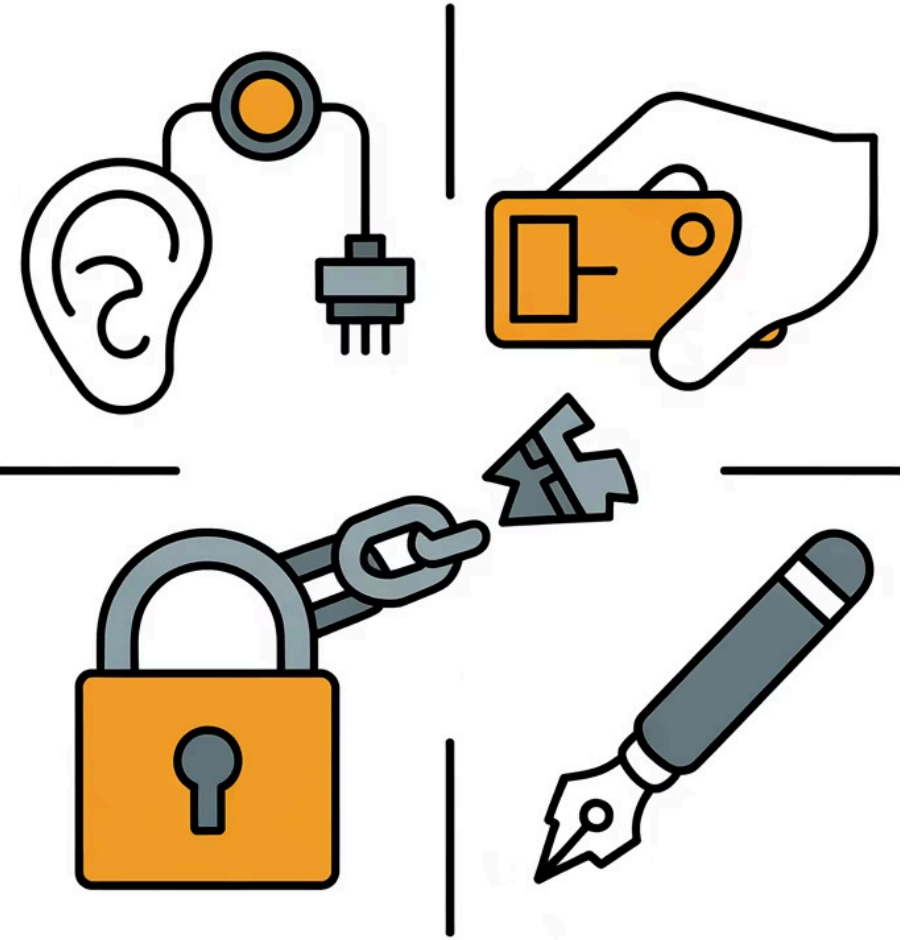
Manipulate and test JWTs including algorithm confusion attacks and signature bypass attempts.

Gobuster

Brute-force discovering of directory/file, DNS and virtual host config.

GitTools

A simple tools to discover, extract and analyse exposed .git directories



Exploitation Principles

Understanding how attackers exploit cryptographic vulnerabilities helps defenders build more robust protections. These core principles underpin most real-world attacks.

Observe

Capture data in transit or find leaked keys through network sniffing, source code analysis, or configuration file inspection.

Tamper

Modify tokens or ciphertext to bypass checks if verification is weak, especially with JWT algorithm confusion attacks.

Crack

Brute-force or use rainbow tables on weak hashes. Modern GPUs can crack millions of MD5 hashes per second.

Impersonate

Forge tokens (e.g., JWT alg:none) or use stolen keys to sign malicious tokens and impersonate other users.

Practical Exploitation Demo

Lab Example: JWT Algorithm None Exploit

Target Application: Web API using JWT tokens

You have a valid JWT token from logging in normally. The application uses JWT for authorization but accepts tokens with "alg":"none" in the header, completely bypassing signature verification.

Exploitation Steps

1. Capture or inspect a legitimate JWT token (optional)
2. Create header {"typ":"JWT","alg":"none"} and payload {"role":"hacker"}
3. Base64url-encode header and payload; join with a trailing dot: <header>.<payload>.
4. Send Authorization: Bearer <header>.<payload>. to a protected endpoint /admin
5. If accepted, demonstrate privilege escalation (view admin page, perform admin action)

Key Insight

This exploitation requires **basic knowledge of JWT structure** and simple Base64 encoding. The attack succeeds because the application fails to validate the algorithm and reject unsigned tokens.

A single configuration mistake can completely bypass authentication and authorization controls.

Why These Vulnerabilities Exist: Root Causes

Understanding why cryptographic failures occur helps development teams prevent them. These fundamental issues appear repeatedly across vulnerable applications.

Lack of Cryptographic Knowledge

Developers misuse algorithms or reimplement crypto. Many breaches aren't due to hackers breaking strong encryption algorithms; rather, they exploit situations where encryption was absent or implemented incorrectly.

Insecure Defaults

Libraries or frameworks using weak defaults or permissive verification. Applications may accept insecure algorithms by default without explicit configuration.

Operational Mistakes

Hardcoded keys, improper key rotation, credentials in CI/CD. Development practices that don't consider security implications of key management.

Performance Shortcuts

Disabling verification for perceived speed gains. Developers may skip expensive cryptographic operations in pursuit of performance.

High-Risk Features and Patterns

Certain application features inherently carry higher cryptographic risk. Security architects should apply extra scrutiny to these areas during design and implementation.

Fast-Moving Teams

Small teams shipping authentication quickly without proper crypto review or using homegrown crypto implementations

Legacy Systems

Legacy systems with old algorithms that haven't been updated or systems integrating many third-party services

Token Sharing Systems

Systems that integrate many third-party services with extensive token sharing between different components

Custom Crypto

Use of homegrown crypto or custom token formats instead of well-reviewed libraries and industry standards

Defensive Strategy: Core Principles

Effective cryptographic security requires a holistic defensive approach built on these foundational security principles. Every secure application must implement these concepts consistently.

Use Well-Reviewed Libraries

Don't roll your own crypto. Use proven encryption algorithms like AES-256 for data at rest and TLS 1.2 or 1.3 for data in transit.

Enforce Encryption

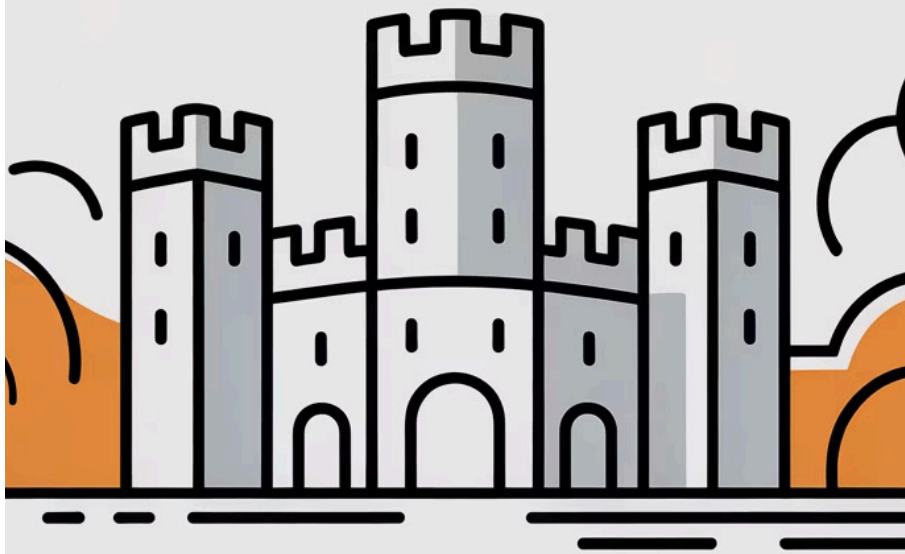
Enforce encryption in transit (TLS 1.2+) and at rest (AEAD modes). Always validate certificate trust chains and disable weak protocols.

Proper Key Management

Idealy use managed services like AWS KMS or Azure Key Vault, at the very least use well secured config files. Avoid hard-coding keys. Enforce key rotation and access controls.

Verify proper configuration

Dont just trust you properly configured tools to not reveal sensitive datas, test it.



Concrete Fixes for Common Cases

Implementing strong cryptographic security requires precise application through code and configuration. This section will provide a brief overview of key areas for mitigation, including robust password storage, stringent TLS enforcement, secure JWT handling, and other critical fixes that are essential for protecting sensitive data and communications.

1. Secure Password Storage

Proper password storage is fundamental to application security. Weak hashing algorithms expose user credentials to various attacks, leading to widespread compromise. Implementing robust, modern hashing functions is non-negotiable for protecting sensitive user data.

Problem: Weak Hashing (MD5/SHA-1)

Weak password hashing algorithms like MD5 or SHA-1 are highly insecure for password storage. They are fast, lack salting, and are vulnerable to rainbow table attacks and brute-force cracking, making it easy to recover original passwords. Even adding a simple static salt does not adequately protect against these attacks.

```php
<?php
// INSECURE password hashing with MD5
$password = "mysecretpassword";
$hashed_password = md5($password);
echo "MD5 Hash: " . $hashed_password . "\n";

// Verification (equally insecure)
$input_password = "mysecretpassword";
if (md5($input_password) === $hashed_password) {
    echo "Password Verified (insecurely)!\n";
} else {
    echo "Password Incorrect.\n";
}
?>
```

Solution: Modern Hashing (Argon2id/bcrypt)

Modern password hashing functions like Argon2id (recommended) and bcrypt are designed to be computationally intensive and resistant to brute-force attacks. They automatically handle salting and have configurable work factors (cost factor, memory, and time), making them much more secure and adaptable to future hardware improvements.

```php
<?php
// SECURE password hashing with Argon2id (recommended)
$password = "mysecretpassword";
$hashed_password = password_hash($password, PASSWORD_DEFAULT);
echo "Argon2id Hash: " . $hashed_password . "\n";

// Verification
$input_password = "mysecretpassword";
if (password_verify($input_password, $hashed_password)) {
    echo "Password Verified (securely)!\n";
} else {
    echo "Password Incorrect.\n";
}
```

2. TLS Enforcement & HSTS

Ensuring that all traffic to your application uses secure TLS (Transport Layer Security) is critical. This involves configuring your server to redirect HTTP requests to HTTPS and implementing HTTP Strict Transport Security (HSTS) to prevent downgrade attacks and ensure browsers always connect securely.

Server Configuration: Redirect HTTP to HTTPS

The first step is to configure your web server to automatically redirect all incoming HTTP traffic to its HTTPS equivalent. This prevents clients from inadvertently connecting over an unencrypted channel. Below is an example for Nginx.

```
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name yourdomain.com www.yourdomain.com;

    ssl_certificate /etc/nginx/ssl/yourdomain.com.crt;
    ssl_certificate_key /etc/nginx/ssl/yourdomain.com.key;

    # ...
}
```

HSTS Header Implementation

HTTP Strict Transport Security (HSTS) is a security mechanism that helps to protect websites against protocol downgrade attacks and cookie hijacking. A web server implements HSTS by providing a response header field named `Strict-Transport-Security`. Once a browser receives this header from a site, it will know to only connect to that site using HTTPS for a specified duration.

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

- **max-age:** Specifies the time (in seconds) that the browser should remember that a site is only to be accessed using HTTPS. A value of 31536000 seconds equals one year.
- **includeSubDomains:** An optional parameter that, if present, applies this HSTS policy to all subdomains of the site as well. This is generally recommended for comprehensive protection.
- **preload:** Not part of the HSTS specification, but an optional flag indicating that the site owner allows the domain to be submitted to a browser's HSTS preload list. This means browsers will enforce HSTS on the domain even before the first visit, offering maximum protection from the start.

3. Secure JWT Handling

JSON Web Tokens (JWTs) are a popular method for securely transmitting information between parties as a JSON object. However, improper implementation can lead to severe vulnerabilities, particularly with verification. Ensuring explicit algorithm specification and robust error handling is paramount to prevent attackers from bypassing signature verification.

Problem: alg:none Bypass

The `alg:none` vulnerability allows an attacker to modify the JWT header to specify `alg:none`, indicating that the token is unsigned. If the server's verification logic does not explicitly check and reject this algorithm for signed tokens, it may accept the token as valid without verifying any signature. This enables an attacker to forge tokens and impersonate users, especially if the payload contains sensitive information like user IDs or roles.

```
// Node.js: dangerous JWT verification with 'faketoken' library
const jwt = require('insecurejwt');

// Use a strong, unique, and securely stored secret key
const secret = process.env.JWT_SECRET

// Auto detect algorithms
const decoded = jwt.verify(token, secret, { algorithms: 'auto' });
```

Solution: Secure Node.js Implementation

To prevent `alg:none` bypass and other algorithm-related vulnerabilities, always explicitly specify the allowed algorithms during verification. Modern JWT libraries enforce this by default or provide options to do so. Rejecting tokens with unexpected or "none" algorithms, combined with proper error handling, ensures that only legitimately signed tokens are processed.

```
// Node.js: Secure JWT verification with 'jsonwebtoken' library
const jwt = require('jsonwebtoken');

// Use a strong, unique, and securely stored secret key
const secret = process.env.JWT_SECRET

// Specify allowed algorithms
const decoded = jwt.verify(token, secret, { algorithms: ['HS256', 'RS256'] }); //
Only allow strong algorithms
```

4. Key Management & Secure Randoms

JSON Web Tokens (JWTs) are a popular method for securely transmitting information between parties as a JSON object. However, improper implementation can lead to severe vulnerabilities, particularly with verification. Ensuring explicit algorithm specification and robust error handling is paramount to prevent attackers from bypassing signature verification.

Centralized Key Management

Effective key management is critical for data security. Centralized services like **AWS Key Management Service (KMS)** and **Azure Key Vault** provide secure storage, generation, and management of cryptographic keys, **OpenBao** offer a suitable free software alternative. These services help enforce least privilege access, audit key usage, and simplify compliance, ensuring that keys are protected throughout their lifecycle and preventing hardcoding or insecure storage in application code.

Data Encryption at Rest (AEAD)

Encrypting data at rest is essential to protect sensitive information stored in databases, file systems, or backups. **Authenticated Encryption with Associated Data (AEAD)** modes, such as **AES-256-GCM**, are highly recommended. AEAD modes not only encrypt the data but also provide integrity and authenticity checks, ensuring that data has not been tampered with and originated from a trusted source. This protects against both unauthorized disclosure and modification.

Cryptographically Secure Randoms

For security-sensitive operations like generating session tokens, encryption keys, or salts, it's crucial to use cryptographically secure pseudo-random number generators (CSPRNGs). Standard functions like `Math.random()` **are often predictable and unsuitable for security purposes**. Always use **cryptographically secure** random generators, such as PHP `random_bytes()`.

Hands-On Practice: CTF Challenge

Account Takeover via exposed secret

Challenge Scenario

You're an IT investigator in charge of investigating a stolen credit card numbers seller known online under the pseudonyme QuartDheure.

QuartDheure started his carrer a few years ago, selling credit cards on a darknet website massively used at the time to buy and sell many illegal products.

The website is mostly considered dead nowadays, but it may still contain usefull informations about your target, particularily the password and email he used back in the day.

Your Mission

1. Find a way to connect as an admin
2. Use the admin secret bitcoin account number (on the admin page) as the CTF key for first chall
3. Use the admin account to recover email/password QuartDheure used back in the days
4. Find the postal address of QuartDheure and use his real name as CTF key

Challenge Details

URL: https://crypto.owasp.ajani.ovh/

Test Accounts:

- alice / password

CTFD URL: https://ctfd.owasp.ajani.ovh/

Expected Duration: 120 minutes

Implementation Checklist

Password Hashing Checklist

- Use a strong, modern hashing algorithm (e.g., bcrypt, Argon2).
- Implement sufficient work factors (iterations, memory, time costs).
- Never store raw passwords.
- Consider password rotation policies.

TLS Configuration Checklist

- Always use HTTPS.
- Enforce TLS 1.2 or higher.
- Enable HSTS (HTTP Strict Transport Security).

JWT Security Checklist

- Specify allowed algorithm (`alg` header) explicitly, never allow `none`.
- Use strong, unique secret keys for symmetric algorithms.
- Rotate keys periodically.
- Store tokens securely (e.g., HTTP-only cookies for session tokens).

Key Management Checklist

- Never store key in code nor add them to repositories.
- Encrypt data at rest using AEAD (e.g., AES-256-GCM).
- Use cryptographically secure random number generators (CSPRNGs).
- Audit key usage regularly.